南開大學
Nankai University

计算机学院

并行程序设计实验报告

Lab1 CPU 架构编程

Cache 与超标量对程序性能的影响

姓名： 柯云超

学号： 2413575

专业： 计算机科学与技术

学院： 计算机学院

2026年3月20日

目录

1. 实验任务 1
2. 实验环境与测试方法 1
   2.1. 实验环境 1
   2.2. 测试数据与规模设计 1
   2.3. 计时方式 2
3. 算法设计与优化思路 2
   3.1. 矩阵列内积 2
   3.2. 求和 2
4. 实验结果与分析 2
   4.1. 矩阵实验 2
   4.2. 求和实验 2
   4.3. 汇编观察与 profiling 结果 3
5. 结论 3

1. 实验任务

本次实验包含两个问题：

1. 给定 $n \times n$ 矩阵 B 和向量 a，计算每一列与向量的内积：

$$s_j = \sum_{i=0}^{n-1} B_{i,j} a_i$$

   对比逐列访问的平凡算法和面向 cache 的优化算法。

2. 给定 n 个浮点数，计算其和：

$$S = \sum_{i=0}^{n-1} x_i$$

   对比链式累加、两路与四路独立累加，以及两两归约等实现。

本次提交的代码不再采用单文件结构，而是按功能拆分为多个模块：`src/benchmark.*` 负责计时和 profiling 框架，`src/matrix_experiment.*` 负责矩阵实验，`src/sum_experiment.*` 负责求和实验，`src/environment.*` 与 `src/profile.*` 负责环境信息和 profiling 输出，入口文件为 `src/main.cpp`。

2. 实验环境与测试方法

2.1. 实验环境

项目	配置
CPU	Intel Core i9-13900H
L1d Cache	48 KiB
L2 Cache	1280 KiB
L3 Cache	24576 KiB
操作系统	Ubuntu 22.04.5 LTS on WSL2
编译器	g++ 12.3.0
编译选项	`-std=c++17 -O3 -march=native`
额外控制	`-fno-tree-vectorize -fno-tree-slp-vectorize`，绑定 CPU0

表 1 实验平台配置

关闭自动向量化是为了把实验重点尽量保留在 cache 局部性和指令级并行本身，而不是让编译器自动生成 SIMD 指令掩盖差异。

2.2. 测试数据与规模设计

两部分实验的数据都使用固定公式生成，便于复现实验并检查正确性：

$$a_i = 0.25 + \frac{(17i + 11) \bmod 97}{97.0}$$

$$B_{i,j} = 0.1 + \frac{(131i + 17j) \bmod 251}{251.0}$$

矩阵实验选取 $n = \{64, 128, 256, 384, 512, 768, 1024, 1536, 2048\}$，工作集大小为 $8n^2$ Byte。求和实验选取 $n = \{2^{10}, 2^{12}, 2^{14}, 2^{16}, 2^{18}, 2^{20}, 2^{22}, 2^{23}\}$，数组大小为 $8n$ Byte，用于观察从 cache 友好到带宽受限的变化过程。

2.3. 计时方式

程序使用 `std::chrono::steady_clock` 计时。对于每组规模，不预先固定执行次数，而是反复运行待测函数直到总计时超过 250 ms，再计算单次平均时间。每次测试前先进行两轮 warm-up，并将线程绑定到逻辑核 0，以减少调度波动。

3. 算法设计与优化思路

3.1. 矩阵列内积

矩阵部分实现了三个版本：

1. **平凡算法**：外层按列循环，内层按行跳跃访问 `matrix[row * n + col]`。这种访问方式与行主存储不一致，空间局部性较差。
2. **行优先算法**：把循环次序改成先按行扫描矩阵，再把该行对所有列的贡献累加到输出向量。这样做不改变计算复杂度，但明显改善了 cache 行利用率。
3. **行优先 + 4 路展开**：在行优先基础上，对内层循环做 4 路手动展开，进一步减少循环控制开销。

3.2. 求和

求和部分实现了四个版本：

1. **链式累加**：单个累加器顺序求和，结构最简单，但依赖链最长。
2. **两路独立累加**：把一条依赖链拆成两条，末尾合并结果。
3. **四路独立累加**：进一步拆成四条独立累加链，提高指令级并行度。
4. **两两归约**：先把相邻元素两两相加，再对中间结果继续归约，直到只剩一个结果。该方法依赖深度更短，但会引入更多中间结果读写。

4. 实验结果与分析

4.1. 矩阵实验

n	平凡/ms	行优先/ms	行优先+U4/ms	加速比 1	加速比 2
64	0.002379	0.001731	0.001223	1.37	1.95
128	0.014189	0.007140	0.005770	1.99	2.46
256	0.062870	0.027437	0.022990	2.29	2.73
384	0.158872	0.061481	0.052697	2.58	3.01
512	0.884909	0.105562	0.093286	8.38	9.49
768	1.445654	0.243319	0.214320	5.94	6.75
1024	5.237621	0.505124	0.422654	10.37	12.39
1536	19.342532	1.876966	1.807951	10.31	10.70
2048	31.570206	3.178951	2.974866	9.93	10.61

表 2 矩阵列内积实验结果

这里的加速比 1 表示“平凡算法 / 行优先”，加速比 2 表示“平凡算法 / 行优先 + 4 路展开”。结果表明：当工作集接近或超过 L2 之后，逐列访问的性能迅速恶化，而行优先版本仍然保持较稳定增长；在 $n = 1024$ 时，行优先 + 4 路展开取得了本次矩阵实验最高加速比 12.39 倍。

4.2. 求和实验

n	链式/ms	两路/ms	四路/ms	归约/ms	两路加速比	四路加速比
1024	0.000616	0.000333	0.000171	0.000533	1.85	3.61
4096	0.002708	0.001341	0.000744	0.002043	2.02	3.64
16384	0.010566	0.005477	0.003521	0.010409	1.93	3.00
65536	0.047305	0.022644	0.014536	0.043753	2.09	3.25
262144	0.181432	0.092731	0.065947	0.338249	1.96	2.75

1048576	0.705655	0.379622	0.305291	1.618618	1.86	2.31
4194304	3.664419	2.697514	2.930423	7.991180	1.36	1.25
8388608	7.133793	5.286948	5.101690	16.656185	1.35	1.40

表 3 求和实验结果

求和实验体现出比较典型的超标量收益曲线。四路独立累加在小规模数据上优势最明显，在 $n = 4096$ 时达到 3.64 倍加速；随着数组规模扩大到超过 L3 的量级，收益逐渐收敛到约 1.3 到 1.4 倍。两两归约虽然缩短了依赖深度，但因为需要反复读写中间结果，在当前实现中整体并不占优。

另外，多链累加和归约都会改变浮点数加法顺序，因此与链式累加存在微小数值差异。实测误差量级在 10^{-13} 到 10^{-10} 之间，属于浮点舍入误差的正常范围。

4.3. 汇编观察与 profiling 结果

从 `objdump -Cd -Mintel` 提取的汇编片段可以直观看到差异：

```
sum_naive:
    vaddsd xmm0, xmm0, QWORD PTR [rdi]

sum_superscalar2:
    vaddsd xmm0, xmm0, QWORD PTR [rax]
    vaddsd xmm1, xmm1, QWORD PTR [rax+0x8]

sum_superscalar4:
    vaddsd xmm0, xmm0, QWORD PTR [rax]
    vaddsd xmm3, xmm3, QWORD PTR [rax+0x8]
    vaddsd xmm1, xmm1, QWORD PTR [rax-0x10]
    vaddsd xmm2, xmm2, QWORD PTR [rax-0x8]
```

链式累加只有一条依赖链，而两路、四路版本分别引入了两个和四个独立累加寄存器，这与实验中的时间结果是一致的。

按照实验指导书的建议，程序中还保留了 `perf_event_open` 的 profiling 代码，尝试记录 cycles、instructions 和 cache miss。但当前环境是 WSL2，硬件计数器不可直接访问，`results/profile_results.csv` 中的所有条目都返回了 `perf_event_open is unavailable in this environment`。因此，本报告对底层原因的判断主要依据两类证据：一是不同工作集规模下的时间变化趋势，二是反汇编得到的目标代码结构。

5. 结论

1. 对矩阵列内积问题，真正决定性能的是访存模式。把逐列访问改成行优先访问后，加速比会随着工作集越过 L2/L3 边界迅速增大，本机最高达到 12.39 倍。
2. 对求和问题，多链累加能够有效利用超标量特性。四路独立累加在小规模数据上达到 3.64 倍加速，但随着数据规模增大，收益会受到内存带宽限制而明显收敛。
3. 两两归约虽然在理论上减少了依赖深度，但在当前实现中引入了额外的中间结果读写，因此整体并不占优。这说明“减少依赖”并不自动等价于“更快”，还要结合访存代价综合判断。

综合来看，本次实验说明了现代 CPU 性能并不只由计算量决定。数据访问顺序、依赖链长度以及编译后机器指令的结构，都会直接影响最终运行时间。